

UserController

1. **POST:** Register new user (/register - hashed password).
2. **PUT/PATCH:** Update user profile details (Name, Email).
3. **PUT/PATCH (2nd):** Change user password/role.
4. **DELETE:** Delete user account.
5. **GET (List + Include):** Get all users with their InstructorProfile (.Include(u => u.InstructorProfile)).
6. **GET (Find):** Get user details by UserId.
7. **GET (Filter):** Filter users by role (Role == "Student").
8. **GET (Sort/Aggregate):** Sort users alphabetically by UserName.

InstructorProfileController

1. **POST:** Create instructor bio profile.
2. **PUT/PATCH:** Update biography text.
3. **PUT/PATCH (2nd):** Clear/reset bio field.
4. **DELETE:** Remove instructor profile.
5. **GET (List + Include):** Get all instructor profiles with User data (.Include(i => i.User)).
6. **GET (Find):** Get profile by InstructorId.
7. **GET (Filter):** Search instructors by bio keyword (.Where(i => i.Biography.Contains(term))).
8. **GET (Sort/Aggregate):** Get total count of registered instructors (.CountAsync()).

CourseController

1. **POST:** Create new course.
2. **PUT/PATCH:** Update course title and price.
3. **PUT/PATCH (2nd):** Update course category (CategoryId).
4. **DELETE:** Delete a course.

5. **GET (List + Include):** Get all courses with Category and Instructor (`.Include(c => c.Category)`).
6. **GET (Find):** Get course details by CourseId.
7. **GET (Filter):** Filter courses by CategoryId or max price.
8. **GET (Sort/Aggregate):** Get top 5 highest-priced courses (`.OrderByDescending(c => c.CoursePrice).Take(5)`).

CategoryController

1. **POST:** Create new course category.
2. **PUT/PATCH:** Update category name.
3. **PUT/PATCH (2nd):** Rename category with slug/code.
4. **DELETE:** Delete empty category.
5. **GET (List + Include):** Get categories including associated courses (`.Include(c => c.Courses)`).
6. **GET (Find):** Get category by CategoryId.
7. **GET (Filter):** Search category by name.
8. **GET (Sort/Aggregate):** Count courses per category (`.Select(c => new { c.CategoryName, Count = c.Courses.Count })`).

ModuleController

1. **POST:** Add module to course.
2. **PUT/PATCH:** Update module name.
3. **PUT/PATCH (2nd):** Change module OrderNumber.
4. **DELETE:** Delete module.
5. **GET (List + Include):** Get modules including lessons (`.Include(m => m.Lessons)`).
6. **GET (Find):** Get module by ModuleId.
7. **GET (Filter):** Get all modules belonging to a specific CourseId.

8. **GET (Sort/Aggregate):** Sort modules by OrderNumber (`.OrderBy(m => m.OrderNumber)`).

LessonController

1. **POST:** Add lesson to module.
2. **PUT/PATCH:** Update lesson title and URL.
3. **PUT/PATCH (2nd):** Update lesson duration (`LessonTime`).
4. **DELETE:** Delete lesson.
5. **GET (List + Include):** Get lessons with parent Module info (`.Include(l => l.Module)`).
6. **GET (Find):** Get lesson by LessonId.
7. **GET (Filter):** Filter lessons by ModuleId.
8. **GET (Sort/Aggregate):** Average duration of all lessons (`.AverageAsync(l => l.LessonTime)`).

EnrollmentController

1. **POST:** Enroll student in course (Triggers enrollment email).
2. **PUT/PATCH:** Update enrollment status ("Active", "Completed").
3. **PUT/PATCH (2nd):** Extend enrollment date.
4. **DELETE:** Cancel enrollment.
5. **GET (List + Include):** Get enrollments with User and Course (`.Include(e => e.User).Include(e => e.Course)`).
6. **GET (Find):** Get enrollment by EnrollmentId.
7. **GET (Filter):** Get enrollments for specific UserId.
8. **GET (Sort/Aggregate):** Count total enrollments per course (`.GroupBy(e => e.CourseId)`).

AssignmentController

1. **POST:** Create course assignment.
2. **PUT/PATCH:** Update assignment title and deadline.

3. **PUT/PATCH (2nd):** Extend deadline date.
4. **DELETE:** Delete assignment.
5. **GET (List + Include):** Get assignments with submissions (`.Include(a => a.Submissions)`).
6. **GET (Find):** Get assignment by AssignmentId.
7. **GET (Filter):** Filter upcoming assignments where `deadline > DateTime.Now`.
8. **GET (Sort/Aggregate):** Sort assignments by closest deadline (`.OrderBy(a => a.DeadLine)`).

SubmissionController

1. **POST:** Submit assignment content.
2. **PUT/PATCH:** Update submission content.
3. **PUT/PATCH (2nd):** Grade submission (SubmissionGrade).
4. **DELETE:** Delete submission.
5. **GET (List + Include):** Get submissions with User and Assignment details (`.Include(s => s.User)`).
6. **GET (Find):** Get submission by SubmissionId.
7. **GET (Filter):** Filter submissions by AssignmentId.
8. **GET (Sort/Aggregate):** Calculate average submission grade for an assignment (`.AverageAsync(s => s.SubmissionGrade)`).

QuizController

1. **POST:** Create quiz for course.
2. **PUT/PATCH:** Update quiz title and total score.
3. **PUT/PATCH (2nd):** Update target CourseId.
4. **DELETE:** Delete quiz.
5. **GET (List + Include):** Get quizzes with student attempt logs (`.Include(q => q.QuizAttempts)`).
6. **GET (Find):** Get quiz by QuizId.

7. **GET (Filter):** Filter quizzes by CourseId.
8. **GET (Sort/Aggregate):** Count total quizzes created per course.

QuizAttemptController

1. **POST:** Log quiz attempt score.
2. **PUT/PATCH:** Update attempt score.
3. **PUT/PATCH (2nd):** Toggle IsPassed flag based on score.
4. **DELETE:** Clear attempt record.
5. **GET (List + Include):** Get attempts with Quiz and User (.Include(qa => qa.Quiz)).
6. **GET (Find):** Get attempt by QuizAttemptId.
7. **GET (Filter):** Get attempts by UserId where IsPassed == true.
8. **GET (Sort/Aggregate):** Get highest attempt score per student (.MaxAsync(qa => qa.QuizAttemptScore)).

CertificateController

1. **POST:** Issue course certificate (Triggers completion email).
2. **PUT/PATCH:** Update certificate code.
3. **PUT/PATCH (2nd):** Update issue date (IssuedAt).
4. **DELETE:** Revoke certificate.
5. **GET (List + Include):** Get certificates with User and Course (.Include(c => c.User).Include(c => c.Course)).
6. **GET (Find):** Get certificate by CertificateId or code.
7. **GET (Filter):** Get all certificates earned by a specific UserId.
8. **GET (Sort/Aggregate):** Sort certificates by latest issued date (.OrderByDescending(c => c.IssuedAt)).